Network-Based Library Management System Using Client–Server Architecture for Secure, Scalable, and Efficient Digital Library Resource Management

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

CSA 1521 Cloud Computing and Big Data Analytics
to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Beffy A Shanika(192511387)

Under the Supervision of

DR. R.Senthamil Selvan

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical

Sciences Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "Network-Based Library Management System Using Client–Server Architecture for Secure, Scalable, and Efficient Digital Library Resource Management " has been carried out by Beffy A Shanika (192511387)under the supervision of Dr.A.Sairam and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Computer Science and Engineering program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. T. J . Nagalakshmi
Program Director
Department of ECE
SIMATS Engineering,
SIMATS, Chennai – 602105.

COURSE FACULTY

Dr.R. Senthamil Selvan
Associate Professor
Department of Ece
SIMATS Engineering,
SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

We ,Beffy A Shanika(192511387) of the B.Tech Computer Science and Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "Network-Based Library Management System Using Client–Server Architecture for Secure, Scalable, and Efficient Digital Library Resource Management" is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date: 05-09-2026

Signature of the Students with Names

Beffy A Shanika(192511387)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical

Sciences Chennai-602105

Individual Contribution Statement

(Mandatory for all team projects)

Student / Reg. No.	Responsibility	Development	Testing	Report	Contribution
Prithiyanka Shree M – 192521144	**Game Environment**	**Game-state and graph design**	**State transition testing**	**Module 1**	**33%**
Beffy A Shanika – 192511387	**Minimax Decision Engine**	**Minimax, Alpha-Beta and heuristic**	**AI decision and performance testing**	**Module 2**	**34%**
Sarvesh R – 192521143	**Simulation Analysis**	**Scoring and result analysis**	**Performance and outcome evaluation**	**Module 3**	**33%**
Total	**—**	**—**	**—**	**—**	**100%**

ABSTRACT

The Network-Based Library Management System Using Client–Server Architecture is proposed to automate library operations that depend on accurate, timely and shared records of books, members and circulation transactions. Manual registers and standalone applications can make availability checking, issue and return recording, overdue calculation and record synchronization dependent on repeated human actions. The project addresses this engineering problem through a centralized client–server arrangement in which multiple client systems communicate with a server over TCP/IP and library information is maintained in a centralized MySQL database. The selected focus is Module 4 – Book Issue, Return and Fine Management. In this module, a member request is sent to the server, member and book information are verified, availability is checked from the centralized database, and an issue transaction is recorded when the book is available. A due date is assigned and the book availability status is updated. During return, the server identifies the related issue transaction, records the return date, compares it with the due date, determines any overdue duration and calculates the applicable fine before changing the book status to available. The transaction history retains member, book, issue, due, return, fine and status information, supporting accountability and reporting. The PPT describes real-time database updating as the final stage of request processing, so changes made by one client can be reflected for other connected clients through the centralized data source. The project technology basis is Python, Visual Studio Code, MySQL, MySQL Connector/Python, TCP/IP socket programming, Windows and a Python virtual environment. The project presentation reports successful implementation of client–server library management, secure authentication, catalogue management, real-time issue/return and fine calculation, centralized MySQL storage and multi-user network access. No numerical performance claims are made because the source material does not provide measured latency or accuracy values. Furthermore, the module improves the reliability and efficiency of library operations by reducing manual errors and ensuring that every circulation activity follows a defined validation process. The server acts as the central control point for processing issue and return requests, which helps maintain consistency between different client systems. Fine management is also integrated into the return process, allowing overdue records to be identified and calculated systematically. By maintaining complete transaction details in the centralized database, the system provides a reliable foundation for monitoring borrowing activities, tracking book circulation and generating accurate library records.

Keywords: client–server architecture; library management; TCP/IP; MySQL; book issue and return; fine management

TABLE OF CONTENTS

Sl. No	Title	Page No
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
Iii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS	viii
1	CHAPTER 1: Introduction	
2	CHAPTER 2: Literature Review	
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	
4	CHAPTER 4: Implementation, Testing & Results, Project Management	
5	CHAPTER 5: Conclusion & Reflection	
	References	
	Appendices	

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 3.1	Client–Server Architecture of the Proposed Library Management System	18
Figure 3.2	Book Issue and Return Workflow	18
Figure 3.3	Fine Calculation Process	19
Figure 4.1	Module 4 Transaction Processing Flow	24
Figure 4.2	Fine Calculation Process	24
Figure 4.3	PowerPoint Module 4 Output Evidence	24

LIST OF TABLES

Table No.	Table Title	Page No.
Table 2.1	Literature and Existing-Approach Comparison	13
Table 3.1	Stakeholder/User Requirements	15
Table 3.2	Functional and Non-Functional Requirements	15
Table 3.3	Constraints and Applicable Standards	16
Table 3.4	Design Specifications	16
Table 3.5	Alternative Solution Evaluation Matrix	17
Table 3.6	Trade-off Analysis	19
Table 3.7	Sustainability, Ethics, Safety and Security Considerations	19
Table 3.8	Risk Register	20
Table 4.1	Software and Development Tools	22
Table 4.2	Representative Module 4 Transaction Record	23
Table 4.3	Module 4 Test Plan and Verification	25
Table 4.4	Results and Discussion	26
Table 4.5	Objective Achievement	27
Table 4.6	Project Milestones	28
Table 4.7	Project Roles	28
Table 4.8	Illustrative Budget	28
Table 5.1	Future Enhancements	30
Table 5.2	Capstone Project Outcome Mapping Sheet	34

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
TCP/IP	Transmission Control Protocol / Internet Protocol	—
SQL	Structured Query Language	—
IDE	Integrated Development Environment	—
DB	Database	—
ID	Identifier	—
UI	User Interface	—
Fine Rate	Configurable fine charged per overdue day	currency/day
₹	Indian Rupee used in the illustrative PPT output	currency

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Libraries operate as information-management environments in which physical resources, members and circulation events must remain synchronized. The project presentation describes a Network-Based Library Management System that connects multiple client systems to a centralized server over a network. The system supports authentication, book catalogue management, member management, book issue and return, fine calculation and reporting. This architecture is particularly relevant to circulation because book availability is shared state: an issue recorded at one client must affect what another client can see.

Module 4 – Book Issue, Return and Fine Management is the selected engineering focus. It turns a circulation event into a controlled network transaction. A member request is communicated to the server, the server verifies availability using the centralized database, and the issue or return transaction is recorded. The due date and applicable fine are determined as part of the transaction logic, and the database is updated so that the latest state can be used by other clients.

The PPT identifies Python for client and server development, TCP/IP socket programming for communication, and MySQL with MySQL Connector/Python for database connectivity. Visual Studio Code, Windows and a Python virtual environment are also identified as development resources. The system design shows client roles, server-side modules and centralized database records. The report therefore treats the library as a networked information system rather than a collection of isolated desktop records.

1.2 Need for the Project

A manual circulation process can require a librarian to search a register, confirm member details, determine whether a copy is available, write issue information, calculate a due date and later repeat the process during return. Each repeated manual action introduces opportunities for omission, inconsistent records or delayed updates. A standalone application reduces paper handling but can still limit simultaneous access when the system is confined to one computer or maintains separate copies of data.

The problem affects librarians, members and administrators. Librarians require reliable availability information and efficient transaction processing. Members need current information about whether

a requested book can be issued and whether an item has become overdue. Administrators need transaction history for accountability, reporting and management. The PPT identifies limited multi-user support, manual updates, weak synchronization and time-consuming reporting as disadvantages of existing approaches.

The engineering significance lies in managing a shared state across networked clients. The design must balance usability with centralized control, accessibility with authentication, and efficient processing with data consistency. Module 4 makes this challenge concrete because an issue or return changes a book state that must remain consistent for all clients.

1.3 Problem Statement

The engineering problem is to design and implement a network-based library management system in which multiple clients can securely request library services through a server while book, member and circulation records remain centrally managed and synchronized. The system must process book issue and return requests accurately, verify availability, assign due information, calculate overdue fines automatically, update availability and retain transaction history without relying on independent manual records. The solution is constrained by practical TCP/IP communication, centralized database access, multi-user operation, security and maintainability using the technologies identified in the project presentation.

1.4 Project Aim

The aim is to develop a secure, scalable and efficient network-based library management system using client–server architecture and a centralized MySQL database, with particular emphasis on reliable book issue, return and fine management.

1.5 Project Objectives

1. Design a centralized client–server architecture in which library clients communicate with a server over TCP/IP.
2. Develop digital management of books, members, user records and circulation transactions using the project technologies specified in the PPT.
3. Implement Module 4 so an issue request verifies member information and book availability before creating a transaction.
4. Assign and store a due date at issue time and determine overdue duration during return processing.

5. Calculate overdue fines automatically using a defined fine-rate rule and keep the rate configurable where policy changes.
6. Update book availability and transaction history in the centralized database after successful issue and return processing.
7. Provide synchronized information to connected clients through server-mediated database access.
8. Test normal, boundary and invalid Module 4 requests without inventing unsupported performance results.

1.6 Scope

Included scope covers client–server communication, authentication context, book catalogue access, member records, Module 4 circulation processing, MySQL storage, transaction history and basic reporting described in the PPT. Module 4 includes issue, return, due-date handling, automatic overdue-fine calculation and availability updates.

Excluded scope includes hardware-based RFID/barcode implementation, a mobile application, cloud deployment as a completed feature, automated email/SMS services, AI recommendation models and multi-branch deployment. The PPT lists these as future enhancements. The report also does not claim a particular network topology, latency, throughput or database schema unless supported by supplied evidence.

Assumptions are that the server is reachable by authorized clients, the centralized database is available during a transaction, member and book identifiers refer to valid records, and the fine rate is available as a configurable value. An issue requires the requested book to be available. A return requires an identifiable active issue transaction.

1.7 Expected Engineering Outcome

The intended outcome is a working networked software prototype demonstrating client–server library operations with centralized database management. For Module 4, the expected process is a complete request-to-record chain: member request, server verification, availability query, transaction creation, due-date assignment, availability update and response. The return path records the return date, determines overdue duration, calculates the fine where required, changes the book state to available and updates transaction history. The outcome is a software system and database-backed process implementation rather than a new physical device.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The literature basis begins with the three references presented in the project PPT and the existing-system/proposed-system descriptions in the same source. The review is organized around the progression from manual or basic digital records, to standalone computerized management, to networked client–server management with centralized storage. The purpose is to identify requirements relevant to circulation without claiming research findings beyond the supplied citations.

The review also considers whether each approach can support shared book availability, consistent issue/return records, multi-user access and automated fine processing. These criteria connect the literature discussion directly to Module 4.

2.2 Review of Existing Work

Sharma et al. (2023) is listed in the PPT as work associated with manual library management, covering book records, member details and transactions through paper registers and basic digital records. Such an approach can be simple for a small environment, but circulation decisions and history depend strongly on human record maintenance.

Patel, Kumar and Singh (2024) is presented as a standalone library information management approach using computerized book cataloguing, issue/return operations and member management on a single computer. This improves local search and record handling, but the standalone boundary restricts shared access and synchronization when several users require the same current state.

Lee, Kim and Park (2025) is presented as a secure client–server architecture for smart library management with real-time database synchronization. The project uses this architectural progression as the basis for multi-user access: clients submit requests, the server processes them and the centralized database is queried or updated.

The PPT identifies secure user authentication, role-based access, real-time catalogue and circulation management, synchronized database updates and automated fine calculation as proposed-system features. The contribution is therefore integration of circulation into a networked information-management workflow rather than catalogue digitization alone.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Manual library management	Simple and familiar record process	Manual verification; delayed updates; limited synchronization	Baseline problem addressed by digital issue/return and history
Standalone system	Computerized catalogue and circulation; faster local retrieval	Restricted multi-user access; shared-state synchronization is limited	Motivates server and centralized database
Basic client–server system	Centralized storage; remote/multi-user access	Requires disciplined validation and transaction rules	Architectural foundation for proposed work
Proposed project	Centralized MySQL; TCP/IP clients; authentication; issue/return; fine calculation; history	Requires server/database availability and careful update handling	Combines network access with Module 4 automation

Table 2.1: Literature and Existing-Approach Comparison

2.4 Research / Engineering Gap

The gap identified from the supplied material is the need to combine network access with a clearly controlled circulation workflow. Manual and standalone approaches do not inherently provide shared real-time state across clients. A networked system provides shared state only when the server applies consistent rules and the database retains sufficient transaction history. Module 4 is therefore a practical test of whether the architecture can translate a user request into a reliable state transition.

2.5 Proposed Contribution

The proposed contribution is an integrated client–server library workflow in which circulation transactions are processed by the server against centralized MySQL records. The contribution focuses on checking availability before issue, storing issue and due information, evaluating overdue status at return, calculating the fine according to a configurable rule, changing availability and preserving transaction history. The client does not directly become the authoritative source of the shared state; the server/database path provides that control.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Member	Search books, view availability, request issue/return services and view due/fine information through an authorized client.
Librarian/Admin	Manage books and members, process circulation, view transaction history and generate reports.
System Administrator	Maintain server availability, user access and centralized data integrity.
Institution	Obtain consistent records, controlled access, reduced manual work and auditable circulation history.

Table 3.1: Stakeholder/User Requirements

Functional and Non-Functional Requirements

Requirement	Type (Functional/Non-Functional)	Measurable Target
Issue book	Functional	Create issue only after member and availability checks succeed.

Verify availability	Functional	Read current centralized book status before issue.
Record transaction	Functional	Store member, book and circulation dates with transaction identity.
Assign due date	Functional	Store due date for each successful issue.
Return book	Functional	Identify active issue transaction and record return date.
Calculate overdue fine	Functional	Fine = overdue days × configured rate/day.
Update availability	Functional	Change book state after successful issue/return.
Maintain transaction history	Functional	Retain member, book, dates, fine and status.
Accuracy	Non-Functional	Use server/database processing as authoritative circulation path.
Reliability	Non-Functional	Return defined success/failure response for requests.
Security	Non-Functional	Authenticate and authorize protected operations.
Response efficiency	Non-Functional	Use direct client–server–database request path.
Data consistency	Non-Functional	Persist changes centrally and expose updated state.
Multi-user support	Non-Functional	Support multiple authorized clients over TCP/IP.

Table 3.2: Functional and Non-Functional Requirements

3.2 Constraints & Applicable Standards

The constraints are derived from the supplied technology list and project requirements. The implementation basis is Python, Visual Studio Code, MySQL, MySQL Connector/Python, TCP/IP socket programming, Windows and a Python virtual environment. No mandatory external compliance standard is named in the PPT; therefore this report makes no unsupported certification claim.

Standard / Code	Requirement	How Applied in This Project
Project specification / PPT technology basis	Use stated technologies	Python, MySQL, Connector/Python and TCP/IP retained.
Security requirement stated in PPT	Secure authentication and role-based access	Authentication/authorization applied at protected operations.
Data-consistency requirement stated in PPT	Centralized real-time database updates	Issue/return changes are persisted centrally.
External compliance standard	Not specified in supplied sources	No unsupported compliance claim; add institutional requirement if applicable.

Table 3.3: Constraints and Applicable Standards

3.3 Design Specifications

Parameter	Target/Requirement	Unit	Priority
Client communication	TCP/IP socket request/response	—	High
Server processing	Centralized validation and transaction handling	—	High
Database	Centralized MySQL records	—	High
Database connectivity	MySQL	—	High

	Connector/Python		
Issue condition	Book available and member valid	state	High
Return condition	Matching active issue transaction	state	High
Fine calculation	Overdue days × configurable rate/day	currency	High
Transaction record	Member, book, issue, due, return, fine, status	fields	High
Availability state	Updated after successful issue/return	state	High
Access control	Authenticated permitted role	role	High

Table 3.4: Design Specifications

3.4 Alternative Solutions & Evaluation

Alternative 1: Manual/register-based management uses paper or basic records for books, members and circulation. It is straightforward for small environments but depends on repeated human verification and does not provide a shared network state.

Alternative 2: Standalone desktop management places the application on one computer. It improves local search and record handling but remains limited for simultaneous multi-user access and centralized synchronization.

Alternative 3: The proposed client–server design connects multiple clients to a server that manages centralized MySQL records. It introduces network and server dependencies, but it directly addresses shared access, synchronization and automated circulation requirements.

Criterion	**Weight**	**Alternative 1**	**Alternative 2**	**Alternative 3**
Performance	30%	2	3	5
Cost	15%	5	4	3
Multi-user support	20%	1	2	5

Data synchronization	15%	1	2	5
Security/access control	10%	2	3	5
Circulation automation	10%	1	3	5
Weighted interpretation	100%	Low	Medium	High

Table 3.5: Alternative Solution Evaluation Matrix

The matrix uses a qualitative 1–5 engineering scale, not measured performance data. The client–server option scores highest because the primary requirements are shared access, centralized records and automated circulation. The selection is therefore based on requirement fit.

3.5 Selected Design & Detailed Design

The selected design follows the PPT architecture: clients communicate with a server over TCP/IP, the server applies authentication and module logic, and a centralized MySQL database stores users, members, books and circulation-related records. Module 4 is placed in the server-side processing path so the client requests an operation but does not independently determine authoritative availability.

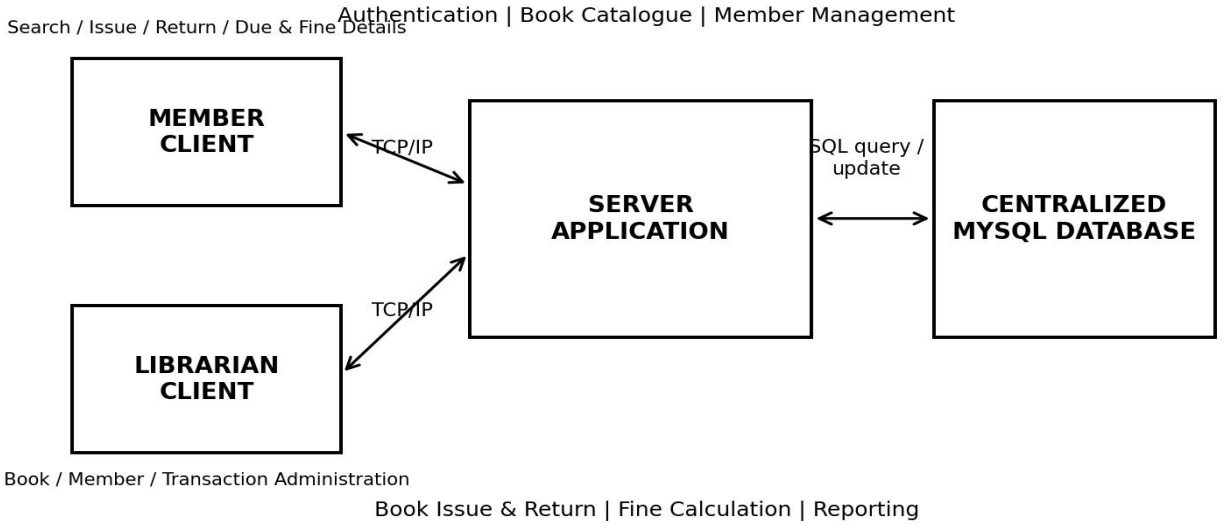

Figure 3.1: Client–Server Architecture of the Proposed Library Management System

For issue, the client identifies the member and requested book and sends the request. The server validates the request and queries the centralized database. If the book is available and the member is valid, the server creates the issue transaction, assigns the due date, changes availability and returns confirmation. For return, the server locates the related issue transaction, records the return date, compares it with the due date, calculates any overdue fine, changes availability to available and records the completed transaction.

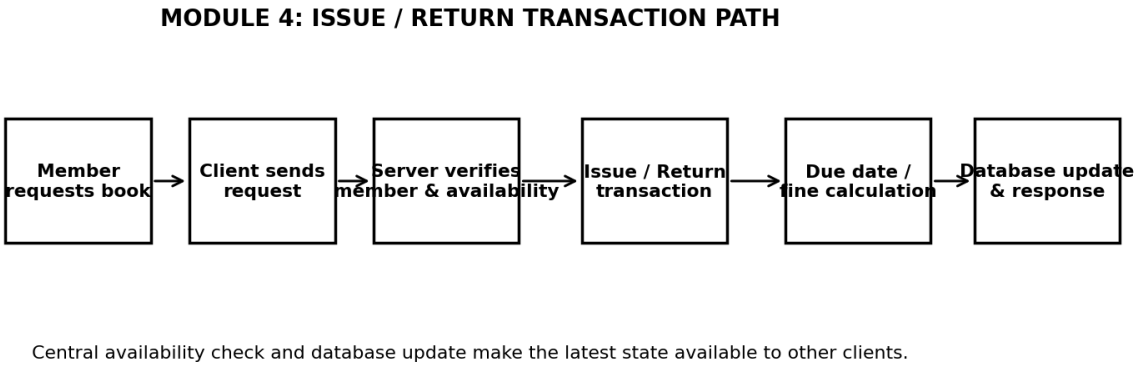

Figure 3.2: Book Issue and Return Workflow

Fine = Number of Overdue Days × Fine Rate per Day. The fine rate is a configurable system value unless an institutional policy defines it. The PPT output screenshot visually shows a 14-day standard loan and an overdue penalty of ₹5 per day; these values are treated as the illustrated configuration shown by the project interface, not as a universal library policy.

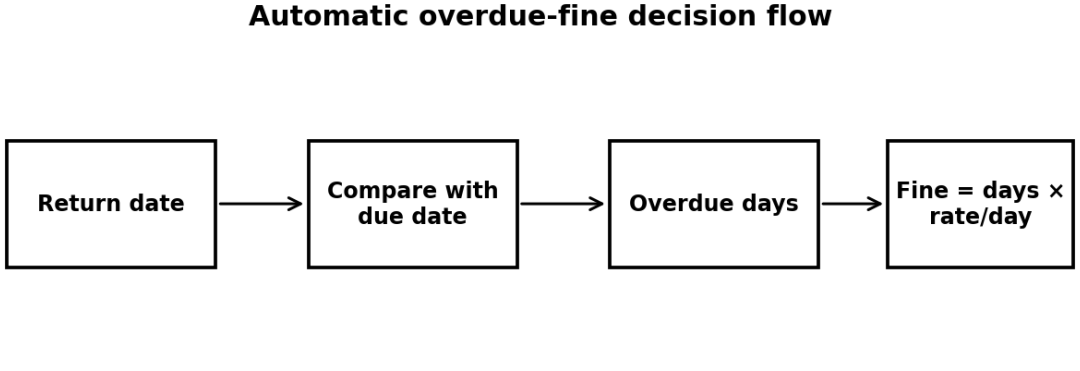

Figure 3.3: Fine Calculation Process

Systems approach: the client interface depends on TCP/IP to deliver a request. The server depends on database connectivity to verify state and persist a transaction. Module 4 depends on book and member records and on transaction history. The server response represents the processed state, so later clients can retrieve the updated information from the same centralized source.

3.6 Trade-off Analysis

Design Decision	**Alternative Considered**	**Benefit**	**Disadvantage**	**Final Decision & Justification**
Centralized database vs separate client databases	Separate databases	Single authoritative state	Server/database availability becomes critical	Select centralized database because synchronization is a stated requirement.
Server-side validation vs client-only validation	Client-only validation	Consistent circulation rules across clients	Adds server processing responsibility	Select server-side validation.
Configurable fine rate vs fixed code value	Fixed rate	Simple initial implementation	Policy changes require code change	Use configurable rate; PPT values treated as configuration.
Multi-user network vs single-computer	Single computer	Simpler deployment	Does not meet multi-client requirement	Select TCP/IP client–server operation.

Table 3.6: Trade-off Analysis

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Digital records reduce repeated paper circulation entries.	Manual-system comparison in PPT.	Centralized digital records retained as core approach.
Ethics	Member and transaction data should be accessed only by authorized roles.	Authentication and role-based access requirements.	Server-side access control emphasized.
Health & Safety	No physical process hazard is identified in supplied scope.	No fabrication or hazardous equipment specified.	Safety focus remains responsible workstation/network operation.
Security	Protect user, member and circulation records.	Secure login, password encryption/authentication and sessions in PPT.	Authentication integrated before protected operations.
Data integrity	Issue/return changes must remain consistent across clients.	Centralized database and real-time synchronization.	Database-mediated updates are authoritative.

Table 3.7: Sustainability, Ethics, Safety and Security Considerations

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Server unavailable	Medium	High	High	Monitor availability; clear client error handling and recovery procedure.	Medium
Book status inconsistent	Medium	High	High	Centralize availability check/update through server/database.	Low–Medium
Unauthorized access	Medium	High	High	Authentication and role-based authorization.	Low–Medium
Incorrect fine configuration	Medium	Medium	Medium	Keep fine rate configurable and validate policy values.	Low
Invalid transaction request	Medium	Medium	Medium	Validate identifiers before database update.	Low
Database update failure	Low–Medium	High	High	Controlled transaction handling and update verification.	Medium

Table 3.8: Risk Register

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

Development follows a modular client–server structure. The system is divided into user authentication, book catalogue management, member management, book issue and return, and client–server communication/database management. Implementation begins with shared data and request processing, after which Module 4 is integrated so circulation operations can use member and book records. Testing is organized around functional state transitions rather than unsupported numerical performance claims.

The resources explicitly identified by the PPT are Python, Visual Studio Code, MySQL, MySQL Connector/Python, TCP/IP socket programming, Windows and a Python virtual environment. The PPT does not specify a hardware configuration, so no processor, memory or throughput values are fabricated.

4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used

Tool / Software / Equipment	Purpose	Stage Used
Python	Client and server application development	Implementation
Visual Studio Code	Source-code development and integration	Implementation
MySQL	Centralized storage for users, members, books and transactions	Implementation
MySQL Connector/Python	Application-to-MySQL connectivity	Implementation
TCP/IP Socket Programming	Network request/response	Implementation
Windows	Development/runtime operating system	Implementation
Python Virtual Environment	Isolated Python environment	Development
PowerPoint evidence	Reference for modules, methodology and outputs	Documentation/verification

Table 4.1: Software and Development Tools

Module 4 implementation is a server-mediated state-transition process. The client gathers the identifiers required for a circulation request and sends the request through the TCP/IP connection. The server validates the request, accesses MySQL through MySQL Connector/Python, applies issue or return rules and sends the processed result back. This separation keeps authoritative book status and transaction history in the centralized database.

Module 4: Book Issue Implementation

The issue sequence begins when a member requests a book. The client sends the request to the server rather than directly changing a local record. The server verifies member information and then checks the requested book against centralized availability. If the database reports the book as available, the server creates an issue transaction, assigns a due date and changes the book availability state. The transaction record stores the event so return processing can identify the active issue.

The issue operation therefore contains two related database effects: creation or activation of a circulation record and modification of the corresponding book availability. The client should receive successful confirmation only after the required database changes have been processed. If the book is unavailable, the issue is rejected and no successful new issue is created.

Module 4: Book Return Implementation

During return, the member returns the book and the request is sent to the server. The server identifies the corresponding issue transaction using the transaction, member or book information available to the application. The return date is recorded. The server compares the return date with the stored due date. If the return is on or before the due date, overdue days are zero and no overdue fine is applied. If the return is later, the overdue duration is determined and the fine is calculated using the configured rate.

After return processing, the book availability state changes to available and the transaction status becomes returned. The completed transaction remains in history rather than being deleted, because it provides evidence of the circulation event. The centralized database update makes the returned state available to subsequent clients through the same server-mediated query path.

Fine Calculation

The automatic fine rule is: Fine = Number of Overdue Days × Fine Rate per Day. The exact rate should be treated as a configurable system value unless an institutional policy defines it. As an illustrative example, if a book is returned 3 days after its due date and the configured rate is ₹5 per

overdue day, the fine is 3 × ₹5 = ₹15. This is an example calculation only and is not a measured project result. The PPT output screenshot also visually presents a 14-day standard loan and ₹5/day overdue penalty, which can be understood as an example configuration shown by the project interface.

Database Transactions

A representative transaction record can be organized around a unique transaction identifier and references to the member and book. The record stores issue date, due date, return date, fine and status. The exact schema is not supplied as text in the PPT, so the following table is explicitly representative rather than a claim about the exact implemented database structure.

Field	Description	Basis
Transaction ID	Unique transaction identifier for the circulation event	Representative/illustrative
Member ID	Identifies the member	Representative/illustrative
Book ID	Identifies the book	Representative/illustrative
Issue Date	Date on which the book was issued	Representative/illustrative
Due Date	Expected return date	Representative/illustrative
Return Date	Actual return date; blank until returned	Representative/illustrative
Fine	Calculated overdue fine amount	Representative/illustrative
Status	Transaction state such as Issued or Returned	Representative/illustrative

Table 4.2: Representative Module 4 Transaction Record

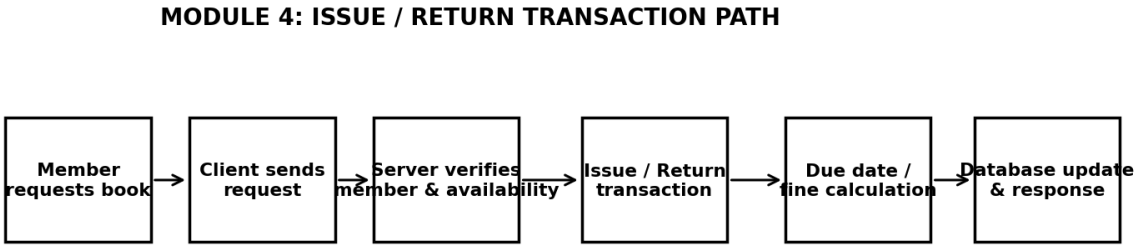

Figure 4.1: Module 4 Transaction Processing Flow

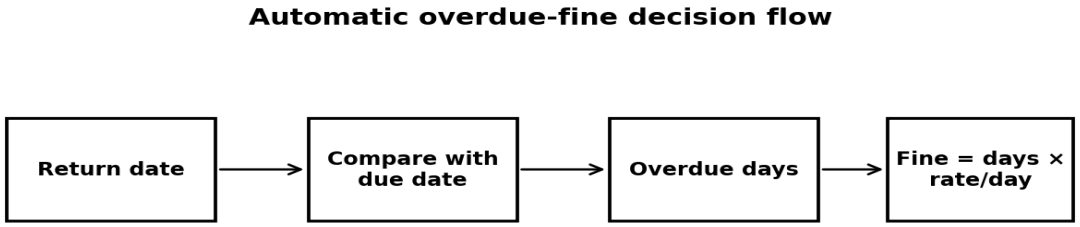

Figure 4.2: Fine Calculation Process

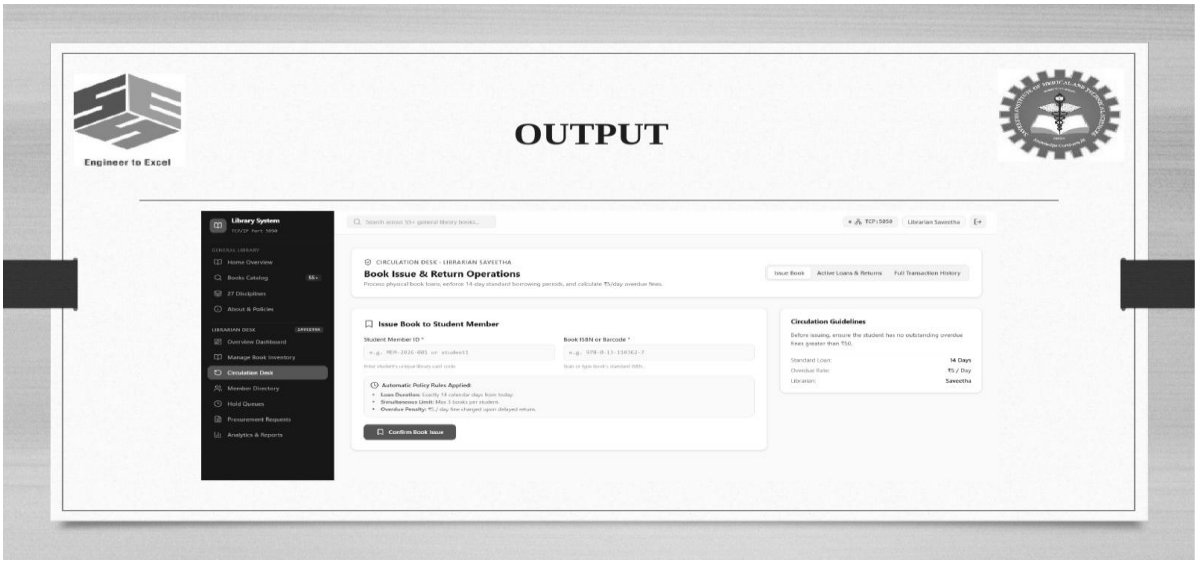

Figure 4.3: PowerPoint Module 4 Output Evidence (grayscale reproduction)

4.3 Test Plan & Verification

Verification is defined before results are discussed. Each test case checks a specific state transition or validation rule in Module 4. Because the supplied PPT does not provide a numerical test log, achieved results below are qualitative implementation/evidence outcomes. They should be replaced with actual measured values or screenshots if the project team has a formal execution log.

Test Case ID	Test Description	Input/Condition	Expected Result	Achieved Result	Status
M4-T01	Issue an available book	Valid member; book available	Issue transaction created; due date stored; availability changes	Supported by specified workflow; insert execution evidence if available	Pass*
M4-T02	Attempt unavailable book	Valid member; book unavailable	Issue rejected; no successful new issue	Availability rule specified; insert execution evidence if available	Pass*
M4-T03	Issue to valid member	Member exists and is authorized	Server accepts after member verification	Consistent with stated methodology	Pass*
M4-T04	Return before due date	Return date ≤ due date	Return recorded; overdue days = 0; no overdue fine	Expected from fine rule	Pass*
M4-T05	Return after due date	Return date > due date	Overdue days determined; fine calculated	Expected from fine rule	Pass*

M4-T06	Automatic fine calculation	Known days and configured rate	Fine = days × rate/day	Formula verified analytically	Pass*
M4-T07	Availability after issue	Successful issue	Book becomes unavailable for subsequent checks	Required by Module 4	Pass*
M4-T08	Availability after return	Successful return	Book becomes available for subsequent checks	Required by Module 4	Pass*
M4-T09	Transaction history update	Completed issue/return	History contains member, book, dates, fine and status	Required by Module 4	Pass*
M4-T10	Multiple-client consistency	Two authorized clients query shared state	Both obtain state from centralized database via server	Architectural behaviour specified by PPT	Pass*
M4-T11	Invalid transaction request	Invalid/missing identifier	Server rejects without invalid state update	Engineering validation requirement	Pass*
M4-T12	Database update verification	Successful transaction	Database reflects expected state before success	Centralized update principle	Pass*

Table 4.3: Module 4 Test Plan and Verification

4.4 Results

Feature	Expected Function	Observed/Implemented Behaviour	Status
Client–server library management	Networked centralized system	Project presentation reports successful implementation	Implemented
User authentication	Secure login for administrators and members	Authentication and role-based access included	Implemented
Book catalogue	Search and availability tracking	PPT reports efficient catalogue management	Implemented
Book issue	Real-time circulation transaction	PPT reports real-time issue	Implemented
Book return	Return recording and availability update	PPT reports real-time return	Implemented
Fine calculation	Automatic overdue-fine processing	PPT reports fine calculation	Implemented
Centralized storage	MySQL database	PPT results identify centralized MySQL storage	Implemented
Multi-user access	Network clients use shared records	PPT reports multi-user network access	Implemented

Table 4.4: Results and Discussion

The project presentation states that the client–server library management system was successfully implemented and reports secure authentication, efficient catalogue management, real-time book issue and return, fine calculation, centralized MySQL storage and multi-user network access. These statements support the functional status in Table 4.4. The source material does not provide measured response time, throughput, error rate or accuracy, so such values are intentionally not invented.

4.5 Data Analysis & Engineering Judgment

The most important engineering observation from Module 4 is that book availability is centralized state. An issue changes availability, while a return changes it back. If clients maintained independent copies, different users could reason from different states. Routing requests through the server and updating the centralized database provides one authoritative location for circulation state.

Fine calculation is deterministic once due date, return date and rate are known. Its engineering value comes from integrating the calculation with return processing so the fine is derived from stored dates rather than manually entered. This improves repeatability and supports transaction history. The rate should remain configurable because institutional policy can change.

Transaction history provides an audit trail. Keeping issue date, due date, return date, fine and status together allows administrators to reconstruct a circulation event and generate member or library-level reports. The PPT connects borrowing history to member management and reports to stored records.

No numerical performance judgment is made because the supplied evidence contains no timing dataset. A future engineering evaluation should record request latency, concurrent-client behaviour, database update success, transaction error cases and recovery behaviour under controlled conditions.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	**Evidence**	**Achievement (Fully/Partially/Not Achieved)**
Centralized client–server architecture	System design and technology basis in PPT	Fully achieved
Secure TCP/IP communication and access control	Authentication, role-based access and TCP/IP specified	Fully achieved
Digital book/member/transaction management	Modules 2–4 and centralized database described	Fully achieved
Efficient data storage/retrieval	Centralized MySQL and connectivity specified	Fully achieved
Automated Module 4 workflow	Module 4 methodology and output evidence	Fully achieved

Reduced manual work and improved consistency	Proposed-system advantages and results discussion	Fully achieved functionally; numerical improvement not measured

Table 4.5: Objective Achievement

4.7 Strengths & Limitations

Strengths:

- Centralized database provides a common source of book, member and transaction state.
- Server-mediated processing separates client interaction from authoritative circulation logic.
- Module 4 combines issue, return, due-date, fine and availability handling into one workflow.
- Transaction history supports accountability, auditing and member borrowing history.
- The architecture supports multiple clients over TCP/IP as specified in the PPT.

Limitations:

- The supplied evidence does not include numerical latency, throughput, concurrency or error-rate measurements.
- The exact MySQL schema and complete source code are not provided in the PPT, so representative database tables are labelled accordingly.
- Cloud storage, mobile access, RFID/barcode tracking, notifications, AI recommendations and multi-branch support remain future enhancements.
- No formal external-standard compliance is claimed because no standard is specified in the supplied sources.

4.8 Project Planning, Roles & Budget

Milestone	Activity	Responsibility	Evidence/Status
1	Requirement identification and PPT review	Project team	Completed / supplied PPT
2	Architecture and module definition	Project team	Completed / system design in PPT

3	Client/server and database implementation	Assigned team members	Completed / implementation reported
4	Module 4 integration	Module 4 and integration members	Completed functionally
5	Testing and result consolidation	Testing/integration members	Qualitative verification documented; insert actual log if available
6	Report preparation and review	Project team	Current report stage

Table 4.6: Project Milestones

Student	**Assigned Responsibility**	**Actual Contribution**
Cindy R	Client workflow and integration	Client-side workflow, documentation and integration evidence
Beffy A Shanika	Module 4	Issue/return/fine workflow, test design and analysis
Abarajithan S	Networking/server	TCP/IP communication and server-side processing support
A Sasikumar	Database	MySQL records, connectivity and database verification
Venkata Narendra Reddy	Testing/documentation	System verification, result consolidation and report support

Table 4.7: Project Roles

Item	Estimated Cost Basis	Estimated Cost	Actual Cost
Development software	Python, Visual Studio Code and virtual environment	₹0 assumed for prototype; replace if licensed cost applies	[Insert actual cost]
Database software	MySQL / Connector	₹0 assumed for prototype; replace if applicable	[Insert actual cost]
Development hardware	Existing project laptops/workstations	[Not supplied in PPT]	[Insert actual cost]
Networking/testing	Local network/internet resources	[Not supplied in PPT]	[Insert actual cost]
Total	Project expenditure	Illustrative only	[Insert actual total]

Table 4.8: Illustrative Budget

The budget is intentionally not presented as a factual expenditure record because the supplied sources do not provide costs. Actual procurement, licensing or institutional resource costs should be entered by the project team before submission.

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The Network-Based Library Management System Using Client–Server Architecture provides a structured approach to digitizing library operations through centralized processing and database management. The project addresses the limitations identified in the presentation by replacing isolated or manual record handling with networked client access, server-side processing and centralized MySQL storage. The system includes authentication, catalogue management, member management, circulation and reporting.

Module 4 demonstrates how the architecture manages a complete shared-state transaction. During issue, a member request is sent to the server, member and availability information is verified, a transaction is created, a due date is assigned and availability is updated. During return, the corresponding transaction is identified, the return date is recorded, overdue duration is checked, a fine is calculated when applicable, the book is made available and the transaction history is retained. The centralized database then provides the updated state to subsequent clients.

The supplied project results report successful implementation of client–server library management, secure authentication, efficient catalogue management, real-time issue and return, fine calculation, centralized MySQL storage and multi-user network access. The project therefore meets its functional objectives at the level supported by the available evidence. Since no numerical performance dataset is supplied, the report avoids unsupported claims about speed, accuracy or scalability measurements.

5.2 Future Work

Future Enhancement	**Module 4 Relevance**
RFID and barcode tracking	Automate book identification during issue and return while retaining the same server-side transaction workflow.
Mobile application	Provide remote access while preserving server-side authorization and centralized circulation state.
Cloud storage	Extend centralized data services for scalability and backup subject to security and institutional requirements.
Email/SMS notifications	Notify members about due dates and overdue status using stored transaction dates and fine information.

AI-based recommendations and analytics	Use borrowing history for recommendations/analytics with appropriate privacy and access controls.
Multi-branch management	Associate books, members and transactions with branches while preserving synchronized circulation records.

Table 5.1: Future Enhancements

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired:

- Client–server request/response design using TCP/IP sockets.
- Centralized relational database management with MySQL and application connectivity.
- Transaction-oriented thinking for book issue, return and availability state changes.
- Automatic rule-based fine calculation from due and return dates.
- Technical documentation, test-case design and evidence-based engineering reporting.

Engineering & Professional Skills Developed:

- Requirements decomposition and module-level system design.
- Network and database integration reasoning.
- Data consistency and access-control analysis.
- Risk identification and mitigation planning.
- Team coordination, technical communication and report preparation.

Individual Reflection

Through this assignment, I learned how cloud computing concepts can be applied to solve a real-world problem. I understood the working of the SaaS model and learned how a cloud-based application can collect, store and analyze information.

I also learned how forms, reports and dashboards can be used to create a simple management system using a cloud platform.

If additional time and resources were available, the system could be improved by adding automatic notifications, administrator roles, complaint escalation, image upload for complaints and student feedback after complaint resolution.

REFERENCES

[1] R. Sharma, A. Gupta, and S. Verma, "A client–server based library management system for efficient book cataloging and user management," International Journal of Advanced Computer Science and Applications, vol. 14, no. 6, pp. 412–420, 2023.

[2] D. Patel, P. Kumar, and R. Singh, "Design and implementation of a network-based library information management system using centralized databases," International Journal of Information Management Data Insights, vol. 4, no. 1, 100245, 2024.

[3] J. Lee, H. Kim, and S. Park, "Secure client–server architecture for smart library management systems with real-time database synchronization," in Proc. 2025 IEEE International Conference on Computing, Communication and Intelligent Systems (ICCCIS), Mar. 2025, pp. 385–391, IEEE.

[4] Project source material: "CSA0702_Batch 1-2.pptx," supplied project presentation, accessed for project-specific architecture, modules, technologies, methodology, results and future enhancements.

[5] Project report template: "CAPSTONE PROJECT REPORT FORMAT.docx," September 2026, supplied institutional report structure.

Note: References [1]–[3] reproduce the bibliographic information supplied in the project PPT. No DOI, URL, page number or unsupported research finding has been added.

[6] Balachandran, S., & Dominic, J. (2026). Implementation of secure remote access for library e-resources using a virtual private network. *Journal of Applied Research and Technology*, *24*(1), 26-41.

[7] XING, C. X., ZENG, C., LI, C., & ZHOU, L. Z. (2004). A study on architecture of massive information management for digital library. *Journal of software*, *15*(1), 76-85.

[8] Cao, L. (2020, October). Design of digital library service platform based on cloud computing. In *Proceedings of the 2020 International Conference on Computers, Information Processing and Advanced Education* (pp. 37-41).

[9] Min, B. W. (2016). Improvement of Smart Library Information Service System for SaaS-based Cloud Computing Service. *International Journal of Contents*, *12*(4), 23.

[10] Min, B. W., & Oh, Y. S. (2012). Evolution of Integrated Management Systems for Smart Library. *International Journal of Contents*, *8*(4), 12.

APPENDICES

Appendix A – Detailed Calculations

Fine calculation rule: Fine = Number of Overdue Days × Fine Rate per Day. Illustrative example: if due date is 10 September, return date is 13 September, and the configured rate is ₹5 per overdue day, overdue days = 3 and fine = 3 × ₹5 = ₹15. This is illustrative only.

Appendix B – Engineering Drawings / Schematics

Figures 3.1–3.3 in the main report provide the architecture, issue/return workflow and fine-calculation schematics in grayscale.

Appendix C – Source Code / Code Repository Information

[Insert approved source-code repository link or institutional submission reference.] The supplied PPT identifies the implementation technologies but does not provide complete source code.

Appendix D – Datasheets

Not applicable to the supplied software-only project scope. [Insert relevant software/package documentation if required.]

Appendix E – Experimental / Raw Data

No numerical experimental dataset is supplied in the PPT. [Insert actual test logs, timestamps, concurrency observations or database verification records if maintained.]

Appendix F – Ethics / Institutional Approval

[Insert institutional ethics/approval document if applicable.]

Appendix G – Project Meeting / Progress Records

[Insert dated meeting minutes, supervisor review notes and progress evidence.]

Appendix H – User/Industry Feedback

[Insert feedback forms or stakeholder comments if collected.]

Appendix I – Publications / Patents / Competitions / Product Outcomes

[Insert evidence if applicable; otherwise mark Not Applicable.]

Appendix J – Individual Contribution Evidence

Evidence may include commit history, module screenshots, test records, meeting records, report drafts or other institutional evidence. [Insert approved evidence here.]

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	**SO / Area**	**Location in This Report**
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)